UNIT 2 · TOPIC 2.1

Cyber Foundations

Assignment, Answer Key

Teacher answer key, 27 points total.

Practice Questions

2.1.a · CIA triad: confidentiality, integrity, availability

1. Which CIA pillar does encryption PRIMARILY protect? (1 pt)

- **● A. Confidentiality**
- ○ B. Integrity
- ○ C. Availability
- ○ D. Authentication

Why: Encryption transforms data so only authorized parties with the key can read it, this is confidentiality. It does not guarantee the data has not been modified (integrity) or that the system is reachable (availability).

2. A ransomware attack encrypts all files on a hospital network and demands payment before restoring access. Which CIA pillar does this PRIMARILY violate? (1 pt)

- ○ A. Confidentiality
- ○ B. Integrity
- **● C. Availability**
- ○ D. Non-repudiation

Why: The attack denies authorized users access to systems and data, that is an availability violation. The files are not leaked (confidentiality) or modified in content (integrity); they are simply made inaccessible.

3. An administrator notices that a financial report was silently edited after it was submitted. Which pillar was violated? (1 pt)

- ○ A. Availability
- **● B. Integrity**
- ○ C. Confidentiality
- ○ D. Redundancy

Why: Unauthorized modification of data is an integrity violation. Hashing and audit logs are the primary controls that would detect this.

4. Which TWO controls are MOST directly used to protect integrity? Select all that apply. (2 pts)

- **■ A. Cryptographic hashing**
- □ B. DDoS mitigation
- **■ C. Access control / audit logs**
- □ D. Content-delivery network

Why: Hashing produces a fingerprint that changes if data is altered, detecting tampering. Audit logs and access control record who changed what and prevent unauthorized writes. DDoS mitigation and CDNs protect availability, not integrity.

5. Describe a scenario where strengthening confidentiality (e.g., full-disk encryption) creates an availability risk. (1 pt)

Acceptable answers: key lost; lose the key; decryption fails; key management; forgotten key; lost key; inaccessible if key; can't decrypt

Why: If the encryption key is lost or the key-management system fails, even authorized users cannot decrypt the data, a confidentiality control has caused an availability failure.

2.1.b · Threat actors and their motivations

6. An attacker deploys ransomware to encrypt university databases and demands a Bitcoin payment in exchange for the key. Which category of threat actor does this most likely represent? (1 pt)

- ● **A. Cybercriminal**
- ○ B. Nation-state actor
- ○ C. Script kiddie
- ○ D. Hacktivist

Why: Cybercriminals are primarily motivated by financial gain. Ransomware attacks that demand payment are classic cybercriminal operations.

7. A quiet, long-duration network intrusion is discovered where custom malware has been slowly copying research emails over eight months. What style of campaign is this? (1 pt)

- ○ A. Script kiddie attack
- ○ B. Hacktivist defacement
- ● **C. Advanced Persistent Threat (APT)**
- ○ D. Accidental insider leak

Why: An Advanced Persistent Threat (APT) is a stealthy, high-capability, long-term campaign typically conducted by state-sponsored actors to gather intelligence or exfiltrate data without detection.

8. Which threat actor group is most likely to target an organization's public-facing website with defacement to promote an ideological cause? (1 pt)

- ○ A. Nation-state actor
- ○ B. Script kiddie
- ○ C. Malicious insider
- ● **D. Hacktivist**

Why: Hacktivists are motivated by ideological or political goals and often conduct loud, highly visible attacks (like website defacement) to generate publicity for their cause.

9. Which of the following are recognized sub-types of insider threats? Select all that apply. (3 pts)

- □ A. Script kiddies using shared credentials
- ■ **B. Negligent employees who accidentally leak files**
- ■ **C. Disgruntled workers seeking revenge**
- ■ **D. Malicious employees acting for financial gain**

Why: Insider threats include malicious insiders (seeking financial or other gains), disgruntled employees (seeking revenge), and negligent insiders (who cause accidental leaks or security failures).

10. What term is used to describe a low-skilled attacker who relies on pre-built, publicly available tools and scripts to launch attacks? (1 pt)

Acceptable answers: script kiddie; script kiddies; script-kiddie

Why: A script kiddie is an attacker who lacks the skill to write custom exploits and instead runs pre-made scripts or software developed by others.

2.1.c · Risk language: likelihood, impact, and security goals

11. An organization decides to buy cyber insurance to cover potential losses from ransomware attacks. Which risk treatment strategy does this represent? (1 pt)

- ○ A. Mitigation
- **● B. Transfer**
- ○ C. Acceptance
- ○ D. Avoidance

Why: Risk transference shifts the financial burden of a risk to a third party, such as an insurance company or outsourced provider.

12. A database containing public marketing materials is deemed to have a low risk score. The IT team decides not to install any additional security controls for it because the cost of controls exceeds the value of the database. What is this strategy? (1 pt)

- ○ A. Mitigation
- ○ B. Transfer
- **● C. Acceptance**
- ○ D. Avoidance

Why: Risk acceptance means acknowledging the risk but taking no active measures to reduce it, which is appropriate for low-value assets where controls are too costly.

13. To prevent unauthorized access to its payment system, a school decides to stop storing credit card data entirely, outsourcing all payment processing to a verified third-party website. What is this strategy? (1 pt)

- ○ A. Mitigation
- ○ B. Transfer
- ○ C. Acceptance
- **● D. Avoidance**

Why: Risk avoidance involves changing plans or processes to completely eliminate the activity that creates the risk, thereby removing the threat entirely.

14. If a security threat has a Likelihood score of 4 (High) and an Impact score of 3 (Medium), what is the calculated Risk Score? (1 pt)

- **● A. 12**
- ○ B. 1
- ○ C. 7
- ○ D. 1.33

Why: Risk Score is calculated by multiplying Likelihood by Impact. Here, 4 * 3 = 12.

15. What term is used to describe any security control that is implemented to reduce either the likelihood or the impact of a risk? (1 pt)

Acceptable answers: mitigation; mitigate; risk mitigation

Why: Mitigation (or risk reduction) is the practice of applying controls to lower the probability or consequences of an incident.

Coding Assignments (Python 3)

2.1.a · CIA triad: confidentiality, integrity, availability

3 pts · auto-graded on the site

Coding problem, 2.1.a: CIA Triad Classifier

Skill: Analyze Risk · **Maps to:** 2.1.A · **Language:** Python 3 (pytest)

Problem

In cybersecurity, classifying incidents according to the CIA triad helps organizations prioritize response and apply correct controls. You are tasked with implementing a rule-based classifier that scans incident descriptions for keywords indicating violations of **Confidentiality**, **Integrity**, or **Availability**.

Implement `solve(data)` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

A list of dictionaries containing `"id"` and `"description"` of the security incidents:

```
[
    {"id": "INC-01", "description": "Attacker intercepted the unencrypted credentials."},
    {"id": "INC-02", "description": "A script corrupted the production database records."}
]
```

Output

A dictionary mapping each incident ID to a sorted list of violated pillars:

```
{
    "INC-01": ["Confidentiality"],
    "INC-02": ["Integrity"]
}
```

Rules

A pillar is violated if the incident's description contains (case-insensitively) any of these keywords:

- **Confidentiality**: `leak`, `expose`, `disclose`, `intercept`, `steal`, `stole`, `read`, `password`, `credential`
- **Integrity**: `modify`, `alter`, `change`, `corrupt`, `tamper`, `edit`, `forge`, `spoof`
- **Availability**: down, `offline`, `unavailable`, `flood`, `crash`, `lock`, `ransomware`, `denial`

The output list of pillars for each incident ID must be sorted alphabetically.

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
from typing import List, Dict, Any

def solve(data: List[Dict[str, Any]]) -> Dict[str, List[str]]:
    """Classifies a list of security incidents into CIA pillars based on keywords."""
    res = {}
    conf_kws = ["leak", "expose", "disclose", "intercept", "steal", "stole", "read", "password", "credential"]
    integ_kws = ["modify", "alter", "change", "corrupt", "tamper", "edit", "forge", "spoof"]
    avail_kws = ["down", "offline", "unavailable", "flood", "crash", "lock", "ransomware", "denial"]

    for item in data:
```

```
        eid = item["id"]
        desc = item["description"].lower()
        violated = []
        if any(kw in desc for kw in conf_kws):
            violated.append("Confidentiality")
        if any(kw in desc for kw in integ_kws):
            violated.append("Integrity")
        if any(kw in desc for kw in avail_kws):
            violated.append("Availability")
        res[eid] = sorted(violated)
    return res
```

2.1.b · Threat actors and their motivations

3 pts · auto-graded on the site

Coding problem, 2.1.b: Threat actor log classifier

Skill: Analyze Risk · **Maps to:** 2.1.B · **Language:** Python 3 (pytest)

Problem

In this challenge, you will implement an automated log classifier for the security team at Meridian University. You are given a list of alert logs, each containing an ID and a text description. You must classify each alert to identify the most likely threat actor category responsible for the attack based on key phrases present in the logs.

Function contract

Implement `solve(logs: list) -> dict` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

A list of log entries:

```
[
    {"id": "ALT-01", "log": "Ransomware deployed. Files encrypted. Payment of 2.5 BTC demanded."},
    {"id": "ALT-02", "log": "FTP scan detected. Automated port scan looking for vulnerabilities."}
]
```

Output

A dictionary mapping log IDs to their threat actor classification:

```
{
    "ALT-01": "Cybercriminal",
    "ALT-02": "Script Kiddie"
}
```

Classification Rules

Match log descriptions (case-insensitive) using the keywords:

- **Nation-State / APT**: `"zero-day"`, `"state-sponsored"`, `"espionage"`, `"sabotage"`, `"rootkit"`, `"lateral movement"`, `"exfiltrat"`
- **Cybercriminal**: `"ransomware"`, `"bitcoin"`, `"payment"`, `"extortion"`, `"payroll"`, `"fraud"`, `"financial"`
- **Hacktivist**: `"protest"`, `"deface"`, `"ideolog"`, `"political"`, `"publicity"`, `"equity"`
- **Insider Threat**: `"employee"`, `"disgruntled"`, `"exit interview"`, `"insider"`, `"negligent"`, `"usb"`
- **Script Kiddie**: `"script kiddie"`, `"coin-miner"`, `"coinminer"`, `"default password"`, `"port scan"`, `"ftp exploit"`, `"automated scan"`

Conflict Resolution: If multiple rules match, prioritize in this order: 1. `Nation-State / APT` 2. `Cybercriminal` 3. `Hacktivist` 4. `Insider Threat` 5. `Script Kiddie`

If none matches, return `"Unknown"`.

Constraints

- Deterministic, offline, standard library only.
- Must pass all cases in `tests.py`.

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
"""Reference solution for 2.1.b. MUST pass 100% of tests.py."""
from typing import List, Dict, Any

CATEGORIES = [
    ("Nation-State / APT", ["zero-day", "state-sponsored", "espionage", "sabotage", "rootkit", "lateral
movement", "exfiltrat"]),
    ("Cybercriminal", ["ransomware", "bitcoin", "payment", "extortion", "payroll", "fraud", "financial"]),
    ("Hacktivist", ["protest", "deface", "ideolog", "political", "publicity", "equity"]),
    ("Insider Threat", ["employee", "disgruntled", "exit interview", "insider", "negligent", "usb"]),
    ("Script Kiddie", ["script kiddie", "coin-miner", "coinminer", "default password", "port scan", "ftp
exploit", "automated scan"]),
]

def solve(logs: List[Dict[str, Any]]) -> Dict[str, str]:
    """Classifies each alert log to identify the most likely threat actor category."""
    res = {}
    for entry in logs:
        lid = entry["id"]
        text = entry.get("log", "").lower()
        classified = "Unknown"

        for cat_name, keywords in CATEGORIES:
            if any(kw in text for kw in keywords):
                classified = cat_name
                break
        res[lid] = classified
    return res
```

2.1.c · Risk language: likelihood, impact, and security goals

3 pts · auto-graded on the site

Coding problem, 2.1.c: Risk Likelihood, Impact, and Decisions

Skill: Analyze Risk · **Maps to:** 2.1.C · **Language:** Python 3 (pytest)

Problem

In cybersecurity, organizations must assess risk and decide how to respond to it. Risk is typically calculated by multiplying the Likelihood of an incident by its Impact: $$\text{Risk Score} = \text{Likelihood} \times \text{Impact}$$

Where both Likelihood and Impact are scored on a scale of 1 to 5.

Once the Risk Score is determined, organizations choose one of four risk treatment decisions: 1. **Avoid**: Eliminate the cause of the risk (e.g., shutting down a highly vulnerable non-critical service). 2. **Mitigate**: Apply controls to reduce the likelihood or impact of the risk (e.g., installing a firewall or implementing MFA). 3. **Transfer**: Shift the risk to a third party (e.g., purchasing

cyber insurance or outsourcing to a secure cloud provider). 4. **Accept**: Decide to live with the risk (usually because the cost of fixing it is higher than the potential impact).

Your task is to write a function `solve(data)` that takes a list of risk scenarios and returns a dictionary mapping each scenario's name to its risk treatment decision based on the following prioritised rules:

1. **Avoid**: If the `risk_score` is 20 or higher AND the mitigation `cost` is strictly greater than 60% of the asset `value` (`cost > 0.6 * value`). 2. **Mitigate (High Risk)**: If the `risk_score` is 15 or higher AND the mitigation `cost` is less than or equal to 30% of the asset `value` (`cost <= 0.3 * value`). 3. **Transfer**: If the `risk_score` is 15 or higher (and does not meet the criteria for Avoid or Mitigate above). 4. **Mitigate (Medium/Low Risk)**: If the `risk_score` is 5 or higher AND the mitigation `cost` is less than or equal to 10% of the asset `value` (`cost <= 0.1 * value`). 5. **Accept**: For all other cases.

Function contract

Implement `solve(data: list[dict]) -> dict[str, str]` in `starter.py`.

Input

A list of dictionaries, where each dictionary represents a risk:

```
[
    {
        "name": "phishing_employees",
        "likelihood": 4,
        "impact": 5,
        "cost": 15000,
        "value": 100000
    },
    {
        "name": "legacy_web_app",
        "likelihood": 5,
        "impact": 4,
        "cost": 70000,
        "value": 100000
    }
]
```

Output

A dictionary mapping the `"name"` of each risk to its decision string (`"Avoid"`, `"Mitigate"`, `"Transfer"`, or `"Accept"`):

```
{
    "phishing_employees": "Mitigate",
    "legacy_web_app": "Avoid"
}
```

Constraints

- Likelihood and Impact are integers in the range `[1, 5]`.
- Cost is a non-negative number (float or int).
- Value is a positive number (float or int, strictly greater than 0).
- Pure function, deterministic, standard library only.

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
def solve(data: list[dict]) -> dict[str, str]:
    results = {}
    for risk in data:
        name = risk["name"]
        likelihood = risk["likelihood"]
        impact = risk["impact"]
        cost = risk["cost"]
        value = risk["value"]

        score = likelihood * impact

        # 1. Avoid: score >= 20 and cost > 0.6 * value
        if score >= 20 and cost > 0.6 * value:
            decision = "Avoid"
        # 2. Mitigate (High Risk): score >= 15 and cost <= 0.3 * value
        elif score >= 15 and cost <= 0.3 * value:
            decision = "Mitigate"
        # 3. Transfer: score >= 15
        elif score >= 15:
            decision = "Transfer"
        # 4. Mitigate (Medium/Low Risk): score >= 5 and cost <= 0.1 * value
        elif score >= 5 and cost <= 0.1 * value:
            decision = "Mitigate"
        # 5. Accept
        else:
            decision = "Accept"

        results[name] = decision
    return results
```